



Software defect prediction model based on LASSO–SVM

Kechao Wang¹ · Lin Liu^{1,2} · Chengjun Yuan¹ · Zhifei Wang¹

Received: 7 February 2020 / Accepted: 17 April 2020 / Published online: 20 May 2020
© Springer-Verlag London Ltd., part of Springer Nature 2020

Abstract

A software defect report is a bug in the software system that developers and users submit to the software defect library during software development and maintenance. Managing a software defect report that is overwhelming is a challenging task. The traditional method is manual identification, which is time-consuming and laborious and delays the repair of important software defects. Based on the above background, the purpose of this paper is to study the software defect prediction (SDP) model based on LASSO–SVM. In this paper, the problem of poor prediction accuracy of most SDP models is proposed. A SDP model combining minimum absolute value compression and selection method and support vector machine algorithm is proposed. Firstly, the feature selection ability of the minimum absolute value compression and selection method is used to reduce the dimension of the original data set, and the data set not related to SDP is removed. Then, the optimal value of SVM is obtained by using the parameter optimization ability of cross-validation algorithm. Finally, the SDP is completed by the nonlinear computing ability of SVM. The accuracy of simulation results is 93.25% and 66.67%, recall rate is 78.04%, and f-metric is 72.72%. The results show that the proposed defect prediction model has higher prediction accuracy than the traditional defect prediction model, and the prediction speed is faster.

Keywords Software defect prediction · Feature selection · Support vector machine · Cross-validation

1 Introduction

Software systems have been spread all over the world, and all aspects of human life have been unable to leave the support of software systems [1]. The software system has propped up the human civilization of the new era and represents new productivity [2]. The software system has changed the way people work, learn, live and think, and is constantly making profound changes to human society and promoting the continuous progress of human civilization [3]. With the development of computer science and software development technology and the continuous advancement of digital information technology, software

systems have played an increasingly important role in human socioeconomic and political life [4]. With the development of technology, software system functions are getting stronger and stronger, software scale is getting bigger and bigger, internal logic relationship is more and more complicated, the number of software defects existing in the system is also increasing, and the difficulty of finding software defects is increasing day by day. [5]. Defects in the software system can cause software system failure, which seriously jeopardizes the security of the software system [6]. However, SDP technology can help software system developers and testers to locate software defects more effectively. Therefore, in-depth research on SDP technology is more and more important in such a background [7].

At present, statistical methods, machine learning and mixed statistics and machine learning have been widely used in the field of SDP [8]. Among these three methods, mixed statistics and machine learning methods have been proven to better achieve SDP. In the field of SDP, because the complexity of software modules, software complexity

✉ Kechao Wang
erickwang@126.com

¹ School of Information Engineering, Harbin University,
Harbin 150086, Heilongjiang, China

² School of Computer Science and Technology, Harbin
Institute of Technology, Harbin 150001, Heilongjiang, China

metrics and the correspondence of error-prone module categories are difficult to determine, researchers have proposed a large number of original complexity metric attribute data for SDP [9, 10]. However, inadvertently introducing all the complexity metric attributes for defect prediction often fails to achieve good results [11, 12]. This is because the excessive introduction of complexity metric attributes leads to “dimension disaster” and “combination explosion,” which leads to an unprecedented increase in the amount of calculation, and the estimation and prediction accuracy will also decrease; in addition, in some cases, some software is obtained. Complexity measurement attribute data is expensive, and if the data itself has little impact on the prediction results, it will inevitably lead to an unnecessarily large increase in the cost of collection of defective data and model application [13, 14]. In view of the above problems, the core of the hybrid statistics and machine learning method is to use the idea of statistical methods to perform feature selection on the data set to be predicted by the defect, so as to reduce the dimension of the data set. Then use machine learning methods to perform defect prediction on the processed data set to obtain the required prediction accuracy [15].

Graph-based regression techniques (such as group additivity) have been widely used to rapidly estimate the thermochemistry of macromolecules. The essence of these techniques lies in the pattern of molecular decomposition. These maps are selected based on heuristics, so they may not provide the best accuracy and it is difficult to choose the electronic effects of non-nearest neighbors such as ring strain, steric hindrance and resonant structure. Here, Frederique explores the feature selection algorithm LASSO to select the best set of graphics descriptors to predict the formation of the standard $\Delta_f H^\circ$. They collected hydrocarbon gas phase data from the NIST Webbook and Burcat databases. They found that models using LASSO-based graphical descriptors from exhaustively enumerated graphics descriptor spaces predicted $\Delta_f H^\circ$ more accurately than traditional group additivity [16]. The smart grid (SG) has become an important form of the Internet of Things (IoT). Despite the promising prospects for renewable energy in Singapore, it poses significant challenges to the existing grids due to its intermittent and uncontrollable power generation characteristics. In order to fully realize its potential, it is indispensable for effective power management to accurately predict the amount of renewable energy generated. Yong proposed a solar power generation prediction model and algorithm based on the minimum absolute contraction and selection operator (LASSO). They compared the proposed solution with two representative solutions with three real-world data sets. They found that LASSO-based algorithms are more accurate than the existing methods, use less training data and are robust to

anomalous data points in training data, and their variable selection capabilities are also provided. Convenience trade-offs and accuracy between complexity, all make LASSO-based proposed methods a highly competitive solution for solar power generation predictions [17]. Based on metrics and deficiencies in software products, SDP techniques can be used to predict more defects that may also exist as early as possible and then appropriately allocate test and verification resources based on the predicted results. Machine learning technology can comprehensively and automatically discover software defects and is becoming one of the main methods of current defect prediction technology. In order to improve the efficiency and accuracy of prediction, the choice and research of machine learning algorithms is the key. Schneidewind compares and analyzes different machine learning defect prediction methods and finds that different algorithms have advantages and disadvantages in different evaluation indicators. Based on the stacked integration learning method, a combined software defect pre-model is proposed. In this model, they first predict that n will add predictions of different methods in the original data set as new software indicators and then predict again. Finally, they experimented on the Eclipse data set. The experimental results show that the model is technically feasible and can reduce time cost and improve accuracy [18]. Based on the existing software defect data, software modules can be predicted by establishing a classification model, thereby improving software testing efficiency and reducing testing costs. Most machine learning-based defect prediction studies are based on two-way decision making methods. SDP can be regarded as a cost-sensitive learning problem, and software data has continuity. Taghi proposes a classification method based on the neighborhood three-way decision theory rough set model. For the ambiguous test module, the experimental results with the NASA software data set show that the method has higher classification accuracy and lower misclassification cost [19]. Software quality is a set of inherent features built into software products throughout the software development process. An important indicator of software quality is the trend of software defects in the life cycle. SDP and software reliability models provide practitioners with an opportunity to observe the distribution of their product defects during development and operations. However, the reported studies have focused on coding or testing. While this is reasonable due to the enforceability of the product, it prevents practitioners from taking advantage of the early identification and prediction of software defects in the life cycle (e.g., reducing costs). Therefore, Kalai outlines the trends in early SDP studies derived from systematic literature searches and elaborates on the methods, attributes and metrics of research involving software process data in defect prediction [20].

Firstly, the feature selection ability of the minimum absolute value compression and selection method is used to reduce the dimension of the original data set, and the data set not related to SDP is removed. Then, the parameter optimization ability of the cross-validation algorithm is used to find the most support vector machine. The simulation results show that the proposed defect prediction model has higher prediction accuracy and faster prediction speed than the traditional defect prediction model.

2 Proposed method

2.1 SDP technology

It is believed that the metric data of the software system describes to some extent the characteristics of the software system. Some of these features are visible and some are invisible. These features are included in the metric data set. Then, there is a logical mapping relationship between software defects and software metric data. SDP technology is to find the content of software defects in these hidden features, to find out whether the software system module has defect information by measuring the implicit logical relationship in the data set.

The main idea of SDP technology is to analyze software metric data using various machine learning methods. Many models and methods have been introduced into the field of SDP. Some representative models are:

- (1) Support vector machine (SVM), verifying the feasibility of using SVM for SDP on NASA data set. The experimental results show that SVM can effectively predict which modules are defective and which modules are not defective and also carry out SVM and comparison of other models shows that SVM is more competitive in the field of SDP. Support vector machine technology performs very well in dealing with small sample problems. Unlike most statistical methods, it does not involve issues such as probability measures and law of large numbers. Any nonsupport vector will not affect the prediction result of SVM, which reduces the computational complexity to a certain extent, so that the support vector machine has good robustness.
- (2) Classification regression tree (CART), the availability of classification regression tree in the field of SDP is discussed. The article proposes a classification rule. You can choose the best balance between the two types of false positive rates and experimentally prove that the classification regression tree can be used for defect prediction. There are many advantages to classifying regression trees. For attribute variables that do not contribute, it can be automatically ignored, providing a reference for reducing variables; it is very suitable for dealing with problems such as the existence of default values or the existence of a large number of variables, and it is fault tolerant. Sexuality can allow partial misclassification of samples.
- (3) Neural network (NN), the feasibility of neural network for SDP is discussed, but the data set used in the literature is its own internal data set, and there is no NASA data set that is used by static prediction technology. Experiments have shown that neural networks can predict software defects. The characteristics of neural network are mainly manifested in three aspects: parallel distributed processing, nonlinear processing and self-learning ability. The highly parallel structure and parallel implementation capability of the neural network enables it to find the optimal solution at a higher speed. It can fully utilize the high-speed computing power of the computer and quickly find an optimal solution. The neural network simulates the nonlinear thinking of the human brain, and its characteristic provides a good help for dealing with nonlinear problems.
- (4) The above three models are the most commonly used models for the development of defect prediction technology, but various model methods have achieved certain results, but also exposed their own shortcomings. Therefore, the research on SDP technology is mainly focused on improving the deficiencies of the existing models. The improvement mainly focuses on two aspects. The essence of the model is mathematical operation, which contains many variable parameters. The model is generally sensitive to parameters. The inappropriate parameter selection method will lead to the reduction of prediction effect. How to reasonably apply to various models? The choice of parameters is one of the problems currently facing. On the other hand, the existence of factors such as data set redundancy makes the classifier inevitably absorb part of the redundancy, which affects the prediction accuracy. How to remove the redundancy factor of the data as much as possible before the data enters the classifier is also one of the problems that have to be solved. Therefore, the current research work focuses on these two issues, improving mature or classic models at two different entry points.
- (5) There is no perfect algorithm in the world, and the algorithm for defect prediction is the same. Other algorithms are introduced to solve the shortcomings of the basic algorithm. At the same time, new problems are introduced, such as the problem of free

parameter uncertainty in the new algorithm and the new parameter pair. The effects of the base model, new parameters cause changes to the underlying model parameters and so on. Therefore, whether the use of the new method to optimize the problem of the basic model can offset the loss caused by the introduction of the new method is also a problem that requires researchers to balance and trade off.

2.2 SDP technology classification

SDP technology has been developed to date and can be divided into static and dynamic defect prediction technologies. The static defect prediction technology can be divided into two categories: classification technology and regression technology. Each type of technology has its own representative model and algorithm, as shown in Fig. 1.

In static prediction technology, support vector machine (SVM) and artificial neural network (ANN) are commonly used models. In dynamic prediction technology, Rayleigh distribution is more commonly used. Gray theory is a rising star and also shows in defect prediction outstanding.

(1) Static prediction technology

The static defect prediction technology mainly predicts the distribution of software defects or the number of predicted defects by analyzing the metric data of the software system. The “80–20” principle of software defects tells us that the distribution of defects in the software system is not uniform. If we can predict which modules belong to “20%,” it will greatly improve the efficiency of software testing and thus improve the software system. Security, this is the goal of static prediction technology. The idea of static prediction techniques is to use machine learning methods to define modules of the system as defective or defect free, to build models and to train models based on known defect data sets and then to use trained models for unknown software system modules. Make predictions to get a prediction of whether the module has defects.

In the early days of static defect prediction technology, the main idea was to estimate the number of defects based on the number of lines of code. With the development of technology, researchers have gradually deepened their understanding of the relationship between defect data and software metrics. More metric attributes have been introduced into the field of SDP, and prediction technology has gradually evolved into metric data using machine learning algorithms. Based on the establishment of the defect

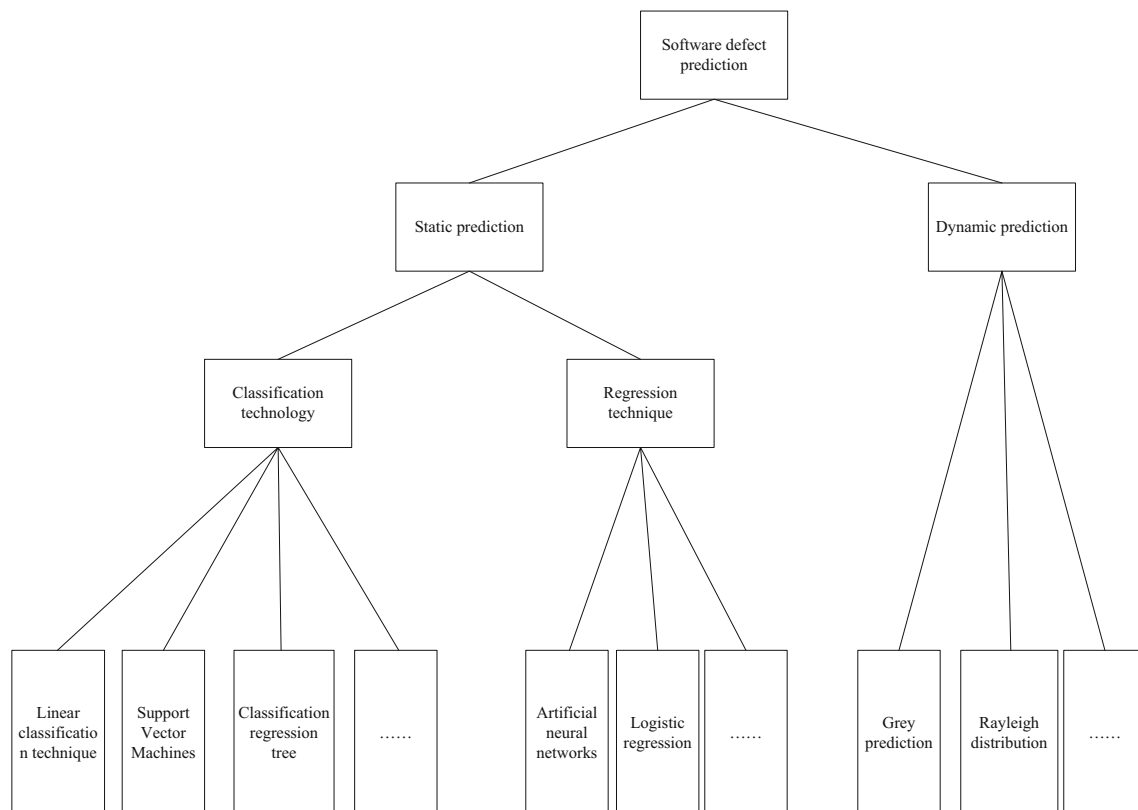


Fig. 1 SDP technology classification

prediction model, the defect prediction technology has also entered the model era. After long-term technological development, the defect prediction technology has matured and many excellent technologies have been born.

(2) Dynamic prediction technology

The entry point of dynamic prediction technology considering the problem is the relationship of software defects with time. The main problem solved by dynamic prediction technology is the trend of software defects with time. Dynamic prediction technology often predicts the number of defects and obtains the number of defects counted in previous software versions. It analyzes the algorithm and predicts the number of software defects that may exist in future versions or maintenance. The idea of dynamic prediction technology is to pay attention to the number of defects that the software system may expose in the future. Dynamic prediction technology can often predict the number of defects that may occur in the next version of the software or the next life cycle stage, so that test and maintenance personnel can work at work. Knowing the approximate bottom line of the number of defects, you can work more purposefully. For example, the number of software defects in a software system during the first N time periods of the life cycle is known. Through dynamic defect prediction technology, the number of defects in the $N + 1$ th, $N + 2$ time periods can be predicted. Knowing the number of defects that may exist in the next period of time, the tester can form a psychological bottom line and will not sloppily end the work when only a small number of defects are found.

Dynamic prediction technology involves time factors. To obtain dynamic prediction data requires long-term monitoring and measurement of software products. Therefore, dynamic prediction technology is more suitable for the operation and maintenance phase of software products, which helps to improve work efficiency in subsequent time periods. The measurement data of static prediction technology is more available in the development stage of software products. Therefore, static prediction technology is often applied to the review and testing stage of software products, focusing on helping software companies improve software quality assurance capabilities and enable software products to Go online in a more complete state. The two prediction technologies have their own emphasis. In practical applications, they should be combined with the actual situation and combined to improve the software quality.

2.3 Related research

(1) LASSO method

LASSO can shrink the regression coefficient of the less-observed observed variable to 0 by the penalty function. Although it sacrifices a certain estimated deviation, it can

reduce the variance of the prediction and improve the accuracy of the prediction.

When using the LASSO method for SDP, it is first necessary to centrally classify the complexity measure attribute data set to eliminate the influence of different indicator dimensions to satisfy:

$$\sum_{i=1}^n y_i = 0, \quad \sum_{i=1}^n x_{ij} = 0, \quad \sum_{i=1}^n x_{ij}^2 = 1 \quad j = 1, \dots, m \quad (1)$$

where m is the dimension and n is the number of variables. Specifically, in SDP, m is the number of attributes of each module in the software complexity metric data set. For example, HAL-STEAD metric, McCabe metric and C&K metric can be used as dimensions, and n is the number of modules in the data set. Suppose a data set consists of 458 modules, each with 42 complexity metric properties, then $m = 42$, $n = 458$.

$$\min S(\cdot)_{x_{j3}} \quad (2)$$

s.t.

$$T(\hat{\beta}) = \sum_{j=1}^m |\hat{\beta}_j| \leq t. \quad (3)$$

Among them:

$$S(\hat{\beta}) = \|Y - \mu\|^2 = \sum_{i=1}^n (y_i - \hat{\mu}_i)^2, \quad \hat{\mu} = X\hat{\beta} = x_{j,j}\hat{\beta}_j. \quad (4)$$

Y_i is the response variable; $x_i = (x_{i1}, x_{i2}, \dots, x_{in})$ is the observed variable; $\hat{\beta}_j$ is the regression coefficient of the j th variable. The LASSO method is to determine the minimum squared sum of the response variable y_i and the regression variable $\hat{\mu}$ by adjusting the regression coefficient of each variable, that is, the value of $\hat{\beta}_j$ under the constraint of $T(\hat{\beta}) \leq t$, that is, the minimum value of $S(\hat{\beta})$ is obtained.

LASSO method In the classification-based SDP model, y_i is a 0–1 attribute variable, which is used to reflect whether the module in the data set is a defective model, and x_i is another complexity measure attribute variable.

Sexual variables The value of t can be $0-\infty$. When the value of t is small, some variable coefficients with low correlation are compressed to 0, so that these variables are deleted to achieve the purpose of feature selection; when t is large enough, it will no longer have the effect of constraint. All properties will be selected and form a variable selection sequence.

(2) Support vector machine

SVM has been widely used in pattern recognition and nonlinear regression. The main idea of the support vector

machine is to establish a classification hyperplane as the decision surface, so that the isolation edge between the positive and negative examples is maximized. The architecture of the support vector machine is shown in Fig. 2.

Among them, K is a nuclear function type mainly:

Linear kernel function: $K(x, x_i) = x^T x_i$ (5)

Polynomial kernel function: $K(x, x_i) = (\gamma x^T x_i + r)^P$ $\gamma > 0$ (6)

Radial basis kernel function: $K(x, x_i) = \exp(-\gamma \|x - x_i\|^2)$ $\gamma > 0$ (7)

Two-layer perceptron kernel function: $K(x, x_i) = \tanh(\gamma x^T x_i + r)$ $\gamma > 0$. (8)

In the above kernel function, the radial basis function (RBF) has a wide convergence range and is an ideal classification basis function. The RBF kernel function is used in the support vector machine classification algorithm.

2.4 SVM algorithm and SDP

This paper introduces the idea of SVM to achieve SDP. The main flowchart of the SDP model based on LASSO-SVM is shown in Fig. 3.

Therefore, there is a certain mapping relationship between the number of defects implied in the software module and the complexity measure attribute. Using the SVM algorithm, some software module attribute values with error tendency are trained to correspond with the error probability of the software module to form a black box model. Using this model to predict the error propensity of the remaining software modules can achieve the purpose of predicting software defects.

Fig. 2 Support vector machine structure

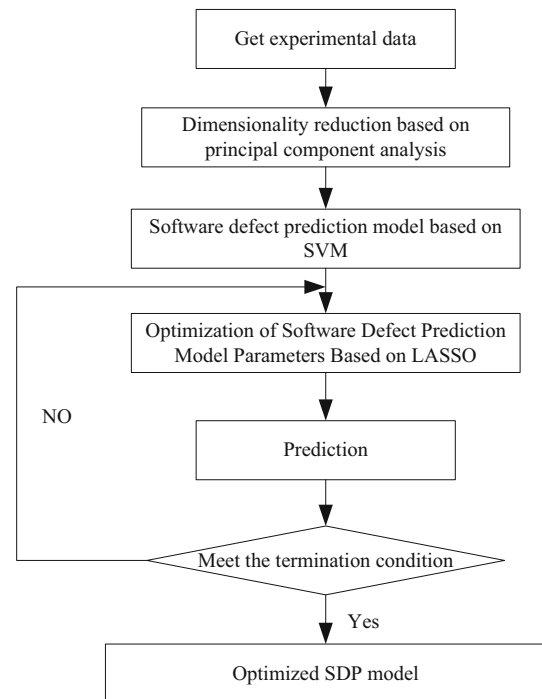
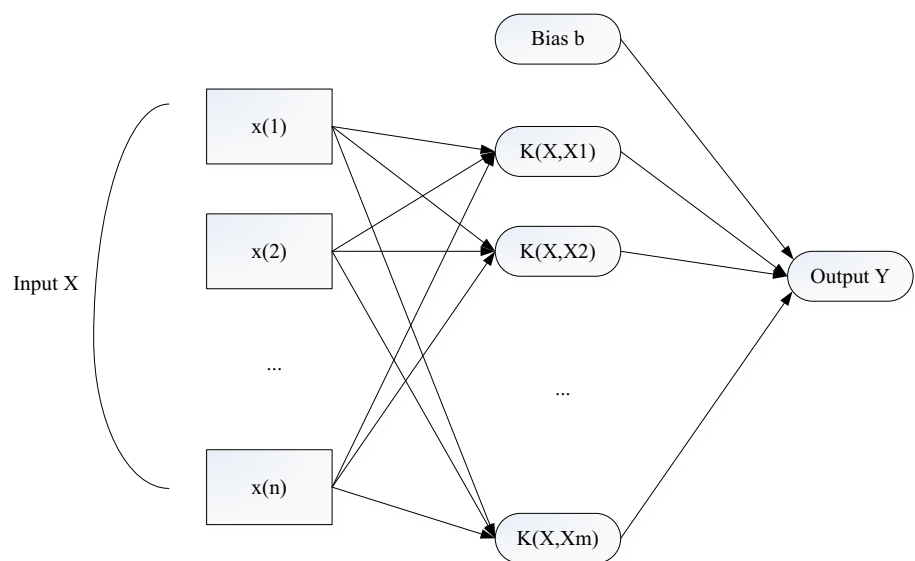


Fig. 3 SDP model flowchart

3 Experiments

3.1 Experimental environment and experimental data

The algorithm used in the experiment was built using the relevant toolbox of MATLAB 2014a. The program runs on a computer configured with Pentium(R)4 CPU 3.00 GHz 2.99 GHz and 504 MB memory.

The experimental data are derived from the PC1 data set in NASA's MDP (metric data program) software metric project, which can be downloaded from a web site. The PC1 data set comes from an aircraft software system, written in C, and contains 1108 subroutines.

3.2 Experimental data preprocessing

NASA's software metric data set has been widely used in research, but few problems may exist in the data set itself. Referring to the latest research results of the school of computer science, University of Hertfordshire, this paper points out that there are a large number of "duplicate data" and "contradictory data" in NASA's data set, which have seriously affected the actual results of software defect prediction.

In this paper, the operation of deleting "duplicate data" and "conflict data" is first carried out on the PC1 data set. After the experiment, the number of subroutines is finally reduced to 953.

Then, the sample data is normalized by the minimum–maximum normalization method, and the value of each attribute of the data sample is mapped to the [0, 1] interval, which eliminates the influence that may be caused by the difference in dimension.

Finally, the LASSO method is used to perform feature reduction on the normalized data, and the attributes that affect the result are eliminated. The SDP model flow is shown in Fig. 4.

3.3 Experimental operation

In this paper, the accuracy, precision, recall and $F1$ values are used to evaluate the predictive power of the SDP model. These metrics are shown in Table 1.

In Table 1: the actual number of positive examples, $P = TP + FN$; the actual number of negative cases, $N = FP + TN$; the total number of instances, $C = P + N$.

- (1) The proportion of the number of modules whose accuracy prediction results are consistent with the actual results in the whole test set.

$$\text{accuracy} = \frac{TP + TN}{C} \quad (9)$$

- (2) Precision (precision): The proportion of modules that are predicted to be error prone in actual error-prone modules.

$$\text{precision} = \frac{TP}{TP + FP} \quad (10)$$

- (3) The recall rate (recall): The proportion of the error-prone module that is correctly identified.

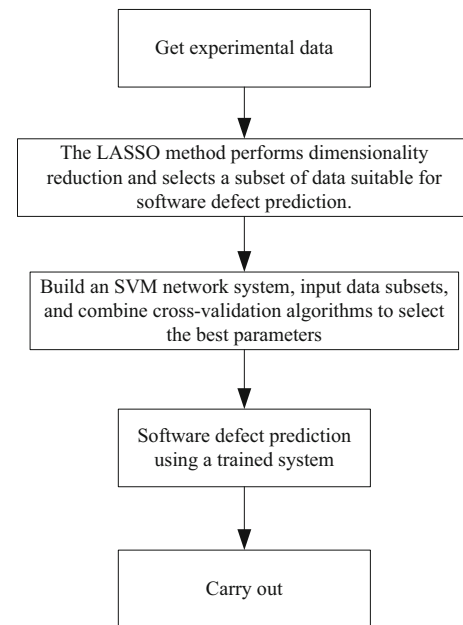


Fig. 4 Flowchart for establishing a SDP model

$$\text{recall} = \frac{TP}{P} \quad (11)$$

- (4) F -measure: The average of the harmonics of recall and precision.

$$F1 = \frac{1}{\frac{2}{\text{precision}} + \frac{1}{\text{recall}}} \quad (12)$$

4 Discussion

4.1 Analysis of experimental results

- (1) Analysis of different training samples

We use precision, recall to evaluate the performance of the classifier. In this experiment, libsvm-3.11 based on the MATLAB platform is used, and the RBF kernel is used. In addition, this experiment uses the accuracy of the tenfold cross-validation test algorithm, that is, the data set is divided into ten equal parts, nine of which are taken as training data in turn, and the other is used as test data to calculate the accuracy or recall rate of each round. Repeat 10 times and calculate the average of 10 times as the final evaluation result.

In order to evaluate the impact of the size of the training data set on the classifier, for each category, we used 100, 200, 300, 400, 500, 600, ..., 2000 number of bug reports as training samples. The average accuracy in the case of different training sample sizes is shown in Table 2.

Table 1 Cross-matrix

Module	Actual value	Predictive value
Error-prone module	Correct positive example (TP)	Correct negative example (FN)
Non-error module	Wrong positive example (FP)	Wrong negative example (TN)

The average accuracy curve for different training sample sizes is shown in Fig. 5.

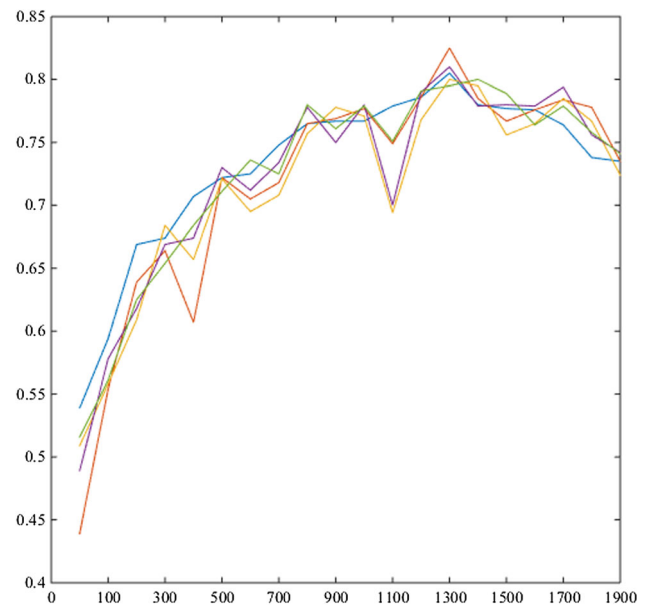
It can be seen from Table 2 and Fig. 5 that for all attribute categories, the accuracy of the bug priority increases with training sample size, especially in the case of the number of samples from 100 to 300, the fastest growth. In all attribute categories, the average accuracy rate is the lowest when the training sample number is 100. When the training sample number is 900, the accuracy rate reaches a certain required height. At 1400, the accuracy rate is the highest. In all attribute categories, when training sample size is 100, the average accuracy is the lowest, indicating that training sample size is 100, which does not meet the training requirements. When training sample size is 1400, the accuracy rate is the highest, indicating that training sample size is 1400, which has reached the requirement of training bug priority classifier. It is not necessary to have a larger number of training samples.

(2) Analysis of training samples of different attribute categories

In addition, the accuracy of the training samples of different attribute categories is shown in Table 3.

The recall rate derived from the training samples of different attribute categories is shown in Fig. 6.

From Fig. 6, it can be seen that when the text attribute TF and the non-text attribute CF are combined, that is, the CSTF, the accuracy and the recall rate of each priority class are improved. This shows that the CF attribute class helps improve accuracy and recall. It can also be seen from the figure that the accuracy and recall rate of the SF attribute category are lower than that of all other attributes. The “Summary” attribute of a bug report has only a few words and contains very little useful information about the priority classification of bug reports. This is why the classifier based on the SF attribute class has the worst performance. In addition, it can be seen from the figure that the accuracy and recall rate are the highest based on the CSTF attribute category. This shows that the combination of non-text

**Fig. 5** Average accuracy for different training samples**Table 3** Accuracy of several bug priority categories based on different training attributes

Attribute accuracy priority	P1	P2	P3	P4	P5
CF	0.535	0.155	0.815	0.244	0.706
SF	0.005	0.030	0.762	0.005	0.002
CSF	0.010	0.201	0.802	0.061	0.656
TF	0.007	0.034	0.764	0.109	0.101
CSTF	0.559	0.209	0.823	0.357	0.712

attributes and text attributes can provide the most useful information and is easy to classify.

(3) Comparison of LASSO–SVM model and single SVM model

The purpose is to prove that the CM1 data set does have data redundancy problem by comparing with the basic model, and the LASSO–SVM SDP model designed in this

Table 2 Accuracy corresponding to different sample numbers

Sample size	100	200	300	400	500	600	700	800	900	1000
Average accuracy	0.539	0.594	0.669	0.674	0.707	0.722	0.725	0.748	0.765	0.767
Sample size	1100	1200	1300	1400	1500	1600	1700	1800	1900	2000
Average accuracy	0.767	0.779	0.786	0.805	0.780	0.777	0.776	0.764	0.738	0.735

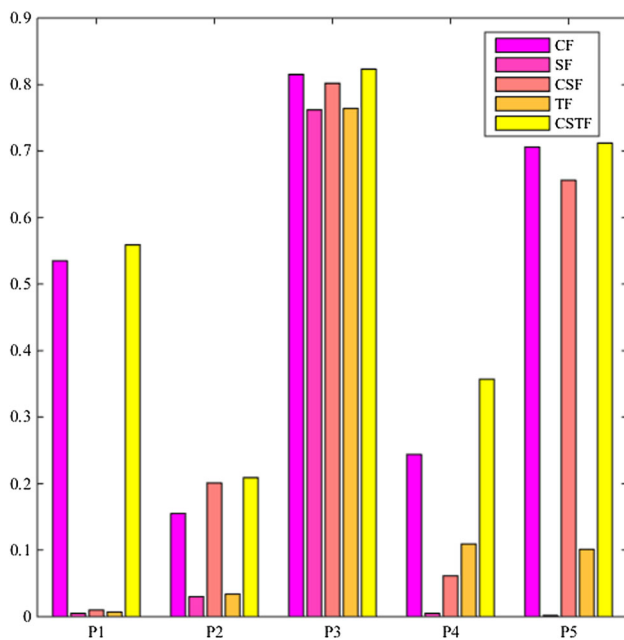


Fig. 6 Accuracy of several bug priority categories based on different training attributes

paper can effectively solve the problem. Using the best experimental results of LASSO–SVM in the experiment, the best prediction results of the LASSO–SVM model and the prediction results of the single SVM model under the same experimental conditions were compared on the four evaluation indicators. The prediction results of a single SVM under the same experimental conditions are shown in Table 4.

Finally, the prediction effects of the two models on the CM1 data set are shown in Table 4, and the results of the experiment 2 are shown in Fig. 7.

4.2 LASSO–SVM compared with other methods

The SDP model based on the method (LASSO–SVM) and Fisher linear discriminant analysis (LDA), cluster analysis (CA), BP neural network (BPNN) and logistic regression (LR) is compared. The results are as follows as shown in Table 5.

The comparison between the indicators of this paper and the traditional method SDP model is shown in Fig. 8.

It can be seen from Fig. 8 that the SDP model based on the LDA method is ineffective because the method is based on the linear classification function for defect prediction, while the software complexity metric attribute values are

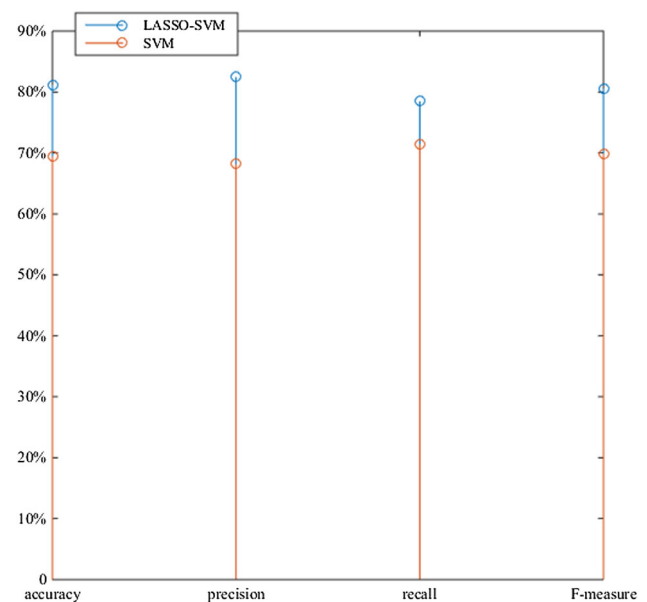


Fig. 7 Comparison of LASSO–SVM and single SVM prediction results

basically nonlinear; based on CA and BPNN, the prediction performance of the method is obviously better than that of the LDA method, but the two methods mainly rely on human experience, and the generalization ability is poor, can have a good predictive effect, and the prediction time is longer.

The method is superior to the traditional SDP algorithm in all indexes. This is because the LASSO feature selection method uses the residual fitting to directly set the regression coefficient affecting the insignificant variable to 0, which reduces the experimental use. Then, the 10 fold cross validation method combined with the attribute subset selected by Lasso is used to optimize the relevant parameters of SVM algorithm, which accelerates the convergence speed and efficiency of SVM network, improves the accuracy of network classification, and well realizes the software defect prediction.

5 Conclusions

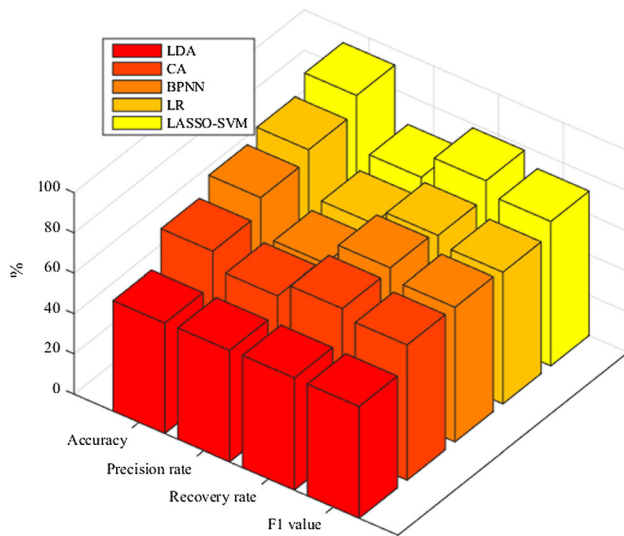
The importance of software products is increasing, and people's attention to software defects has also increased. SDP technology can help software developers and testers locate software defects more accurately, allocate limited resources more appropriately to appropriate software

Table 4 LASSO–SVM and single SVM experimental data

LASSO–SVM	Accuracy (%)	Precision (%)	Recall (%)	F-measure (%)
	78.26	79.4	75.46	79.85
SVM	71.32	69.29	69.25	70.64

Table 5 Comparison of the method in this paper with the traditional method

Predictive model	Accuracy (%)	Precision rate (%)	Recovery rate (%)	F1 value (%)
LDA	54.9	55.3	55.6	55.4
CA	71.6	63.5	71.2	67.1
BPNN	79.3	60.6	72.4	66.9
LR	84.2	61.5	69.6	65.3
LASSO-SVM	92.14	65.6	77.9	71.6

**Fig. 8** Comparison of the method of this article and the traditional method

modules and improve software quality. In this paper, by analyzing the characteristics of the software defect data set, the LASSO method is used to achieve data dimensionality reduction. The software defect data in the high-dimensional Euclidean space is reconstructed in a low-dimensional space, and the manifold structure of the software defect data in the high-dimensional Euclidean space is retained, thereby obtaining the corresponding embedded map, thereby realizing the data. The support vector machine (SVM) is used as the basic classifier, and the SVM is trained with the reduced-dimensional data. The SDP model of LASSO-SVM is given.

In order to verify the feasibility of the LASSO-SVM-based SDP model, this paper establishes the LASSO-SVM model and designs the experiment to verify the prediction effect. Firstly, through the comparison of different embedding dimensions, the optimal embedding dimension of LASSO-SVM model on NASA CM1 data set is obtained. The prediction result under optimal embedding dimension is taken as the representative result of LASSO-SVM model, and LASSO is carried out. SVM SDP model and single SVM SDP model are compared and verified on CM1 data set. The results show that LASSO-SVM SDP model can solve the data redundancy problem in the field

of SDP, and the prediction accuracy is simple. SVM is better.

This paper uses the LARS algorithm to solve the problems involved in the LASSO method. The LARS algorithm is suitable for low-dimensional data space. If you encounter a software complexity metric attribute set that is much larger than the test history data, i.e., $m \gg n$, in this case, you need to find the rest of the algorithm to deal with the LASSO problem. There is a problem when cross-validation selects the SVM optimal parameters, that is, there may be multiple sets of parameters corresponding to the highest verification classification accuracy rate. The treatment measures in this case are the next research direction.

Acknowledgements This work was supported by Natural Science Foundation of Heilongjiang Province (Grant No. LH2019F046), Harbin science and technology innovation talents research project (Grant No. 2016RAQXJ013) and Doctoral research fund of Harbin University (Grant No. HUDF2019101).

Compliance with ethical standards

Conflict of interest The authors declare that they have no conflict of interest.

References

- Mao F, Li B, Shen B (2016) Cross-project software defect prediction based on instance transfer. *J Front Comput Sci Technol* 10(1):43–55
- Wan H, Wu G, Yu M (2019) Software defect prediction based on cost-sensitive dictionary learning. *Int J Softw Eng Knowl Eng* 29(9):1219–1243
- Lee T, Nam J, Han D (2016) Developer micro interaction metrics for software defect prediction. *IEEE Trans Softw Eng* 42(11):1015–1035
- Zhang Z-W, Jing X-Y, Wang T-J (2016) Label propagation based semi-supervised learning for software defect prediction. *Autom Softw Eng* 24(1):1–23
- Nam J, Fu W, Kim S (2018) Heterogeneous defect prediction. *IEEE Trans Softw Eng* 44(9):874–896
- Yu X, Liu J, Peng W (2017) Improving cross-company defect prediction with data filtering. *Int J Softw Eng Knowl Eng* 27(10):1427–1438
- Tantithamthavorn C (2017) An empirical comparison of model validation techniques for defect prediction models. *IEEE Trans Softw Eng* 43(1):1–18
- Herbold S (2017) Comments on ScottKnottESD in response to “An empirical comparison of model validation techniques for

- defect prediction models". *IEEE Trans Softw Eng* PP(99):1091–1094
9. Li F, Lu Y (2016) Lasso-type estimation for covariate-adjusted linear model. *J Appl Stat* 45(1):1–17
 10. Hong L, Dai F, Liu H (2016) A fused-lasso-based doppler imaging algorithm for spinning targets with occlusion effect. *IEEE Sens J* 16(9):3099–3108
 11. Radmanesh N, Burnett I, Rao B (2016) A lasso-LS optimization with a frequency variable dictionary in a multizone sound system. *IEEE/ACM Trans Audio Speech Lang Process* 24(3):583–593
 12. Das R, Walia E (2019) Partition selection with sparse autoencoders for content based image classification. *Neural Comput Appl* 31:675–690
 13. Guo J, Chen Z, Ban Y-L (2017) Precise enumeration of circulating tumor cells using support vector machine algorithm on a microfluidic sensor. *IEEE Trans Emerging Top Comput* 5(99):518–525
 14. Peng X (2019) A spheres-based support vector machine for pattern classification. *Neural Comput Appl* 31:379–396
 15. Manavalan B, Shin TH, Lee G (2018) DHSpred: support-vector-machine-based human DNase I hypersensitive sites prediction using the optimal features selected by random forest. *Oncotarget* 9(2):1944–1956
 16. Lang F, Chavarro D, Liu Y (2016) Can automatic classification help to increase accuracy in data collection? *J Data Inf Sci* 1(3):42–58
 17. Yong H, Zhang X, Feng B et al (2015) iTrade: a mobile data-driven stock trading system with concept drift adaptation. *Int J Data Wareh Min* 11(1):66–83
 18. Schneidewind NF (2015) Experience with risk-based software defect prediction models. *IEEE Trans Syst Man Cybern B Cybern A Publ IEEE Syst Man Cybern Soc* 40(4):997–1008
 19. Khoshgoftaar TM, Gao K, Chen Y (2015) Comparing feature selection techniques for software quality estimation using data-sampling-based boosting algorithms. *Int J Reliab Qual Saf Eng* 22(03):356–13
 20. Kalai Magal R, Jacob SG (2015) Improved random forest algorithm for software defect prediction through data mining techniques. *Int J Comput Appl* 117(23):18–22

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.